

## Problem C. Line Town

The final happiness of a resident depends only on the parity of the displacement between the initial and final positions. If the final arrangement is  $p$ , the value at final position  $i$  is  $h'_i = (-1)^{i-p_i} h_{p_i}$ . We need the minimum number of adjacent swaps, equivalently inversions of  $p$ , such that  $h'$  is nondecreasing.

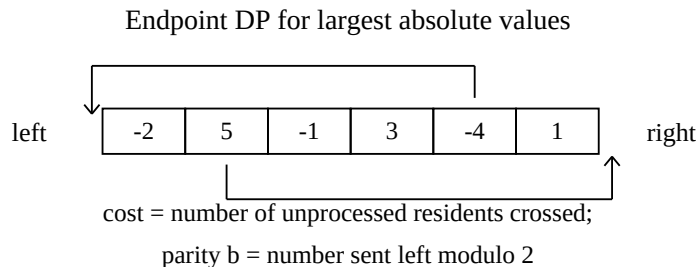


Figure 1. The largest remaining absolute value must go to one of the two endpoints.

### Subtasks 5 and 6 – distinct absolute values via endpoint DP

Process residents in decreasing order of  $|h|$ . The largest remaining absolute value must end at the left or right endpoint of the remaining interval; the sign constraint may allow one, both, or neither endpoint. Let  $dp[k][b]$  be the minimum number of swaps after placing the  $k$  largest absolute values, where  $b$  is the parity of how many have gone to the left. If the  $k$ -th resident crosses  $\ell_k$  already-placed residents to go left, transition to  $b \oplus 1$  with cost  $\ell_k$ . If it crosses  $r_k$  to go right, stay at  $b$  with cost  $r_k$ . Subtask 5 computes these costs naively in  $O(N^2)$ ; subtask 6 computes them with a Fenwick tree in  $O(N \log N)$ .

### Subtasks 1 and 2 – values are only $\pm 1$

Flip the sign of every other position. After this transformation, adjacent swaps no longer change signs. There are  $N + 1$  possible final splits between  $-1$  and  $+1$ . For a fixed split, match the positions of  $+1$  in the initial and final strings in order; the cost is the sum of absolute differences. Subtask 1 enumerates all splits independently in  $O(N^2)$ , while subtask 2 sweeps the split and maintains the sum after only one target position changes, giving  $O(N)$ .

### Subtasks 3 and 4 – allow zeros among a group

Again flip alternating positions. The task for one candidate final configuration becomes transforming one array over  $\{-1, 0, 1\}$  into another. Label all  $-1$ s left-to-right, then all  $0$ s, then all  $+1$ s. The minimum adjacent swap count is the inversion count between the two label permutations. Subtask 3 computes the inversion count for every candidate configuration independently; subtask 4 sweeps configurations and maintains the changed inversion contribution.

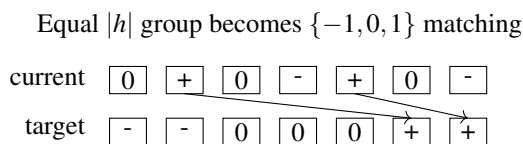


Figure 2. Equal-absolute-value groups reduce to a three-symbol matching problem.

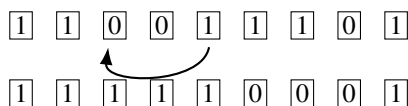
### Subtasks 7 and 8 – full solution

Process equal- $|h|$  groups together. Larger absolute values already placed determine the current endpoints and parity. For the current group, use the three-symbol matching machinery to compute the cost of sending each feasible number of group elements to the left, then merge these costs into the endpoint DP. This combines group matching with endpoint DP while charging each inversion exactly once.

## Problem D. Flip it and Stick it

For every possible target string  $T$ , characterize the strings that avoid  $T$ . Call such strings good. The solution defines a potential  $f(S)$  such that good strings have  $f = 0$ , one reversal can reduce  $f$  by at most one, and a greedy reversal always reduces it by exactly one when  $S$  is not good. By bit-flip symmetry, assume  $T$  starts with 0.

One reversal reduces a potential by one



Example for  $T = 01$ : move the first internal 1-block into the prefix block.

Figure 3. A typical greedy reversal merges a bad block into a good prefix.

### Subtask 1 – $T = 0$

A good string contains only 1s. Therefore the answer is 0 if  $S$  has no 0, and  $-1$  otherwise.

### Subtask 2 – $T = 01$

Good strings have the form  $1^*0^*$ . The answer is the number of 1-blocks that are not already the prefix block, equivalently the number of occurrences of substring 01. Greedily reverse from the first internal 1-block to merge it with the prefix.

### Subtask 3 – $T = 00$

Good strings have no 0-block of length at least two. If the number of zeros is larger than the number of ones plus one, no rearrangement can be good and the answer is  $-1$ . Otherwise, the answer is  $\sum(\text{block size} - 1)$  over all zero-blocks. A reversal cuts a 0 from a long zero-block and inserts it between a pair of 1s.

### Subtask 4 – $T = 001$ or $T = 011$

By reversal and bit-flip symmetry, solve  $T = 011$ . Good strings have a prefix block of 1s and no later 1-block of size at least two. The answer is the number of non-prefix 1-blocks of length at least two.

### Subtask 5 – $T = 010$

Good strings have no isolated 1-block. If there are  $x$  isolated 1-blocks, one reversal can merge up to two of them, and the answer is  $\lceil x/2 \rceil$ .

### Subtask 6 – $T = 000$

Let the 1s split  $S$  into zero-block lengths  $x_1, x_2, \dots, x_k$ , allowing zero-length gaps. A good state has every  $x_i \leq 2$ ; if the number of zeros is larger than twice the number of ones plus two, the answer is  $-1$ . Without empty gaps, the cost would be  $\sum_{x_i > 2} (x_i - 2)$ . Empty gaps can save reversals by cutting off two zeros at a time, so

$$f(S) = \sum_{x_i > 2} (x_i - 2) - \min \left( \sum_{x_i > 2} \left\lfloor \frac{x_i - 2}{2} \right\rfloor, \#\{i : x_i = 0\} \right).$$

The greedy uses a size-zero gap when possible, otherwise a size-one gap.

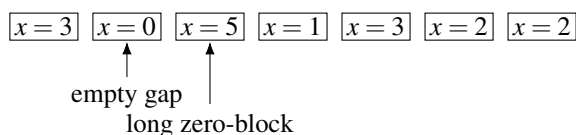


Figure 4. For  $T = 000$ , reversals operate on zero-block sizes.

## Problem E. Travelling Trader

The three values  $K = 1$ ,  $K = 2$ , and  $K = 3$  lead to different structures. The official solution presents them in the order  $K = 1$ ,  $K = 3$ , and  $K = 2$ .

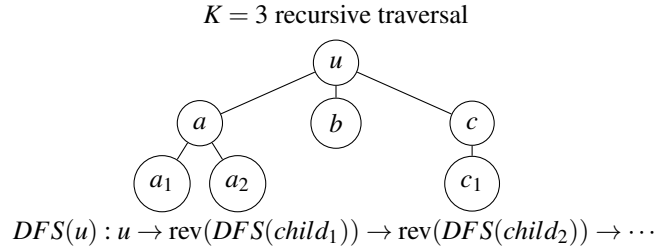


Figure 5. For  $K = 3$ , recursively concatenate reversed child traversals.

### Subtask 1 – $K = 1$

The trader can follow only one root-to-node path. The answer is the maximum node-weighted path starting at the root, computed by one DFS.

### Subtasks 5 and 6 – $K = 3$

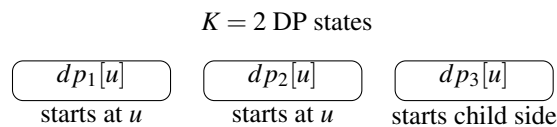
When  $K = 3$ , all nodes can be visited. Define  $DFS(u)$  to start at  $u$ , visit every node in the subtree of  $u$ , and end at a child of  $u$  (or at  $u$  for a singleton subtree). If  $v_1, \dots, v_k$  are the children of  $u$ , concatenate

$$u, \text{rev}(DFS(v_1)), \text{rev}(DFS(v_2)), \dots, \text{rev}(DFS(v_k)).$$

Naively copying and reversing path arrays is quadratic. Small-to-large merging of deques, or an auxiliary graph of path pieces traversed at the end, gives efficient reconstruction.

### Subtasks 2, 3, and 4 – $K = 2$

Use DP on the rooted tree. Let  $dp_1[u]$  be the best path starting at  $u$  and staying in  $u$ 's subtree. Let  $dp_2[u]$  start at  $u$  and end either at  $u$  or at a child-side endpoint. Let  $dp_3[u]$  start at a child-side endpoint and stay in  $u$ 's subtree. The transitions combine child contributions in a few patterns, so keeping the best few child expressions lets each transition be evaluated in constant time per child.



Only a constant number of child subtrees are special in each transition; keep top candidates.

Figure 6. The three  $K = 2$  states and their child-side endpoints.

## Problem F. Triangle Collection

Every triangle either uses an equal-length pair as two sides plus one remaining base, or is formed from leftover pairs after all useful odd sticks have been handled. The official solution starts from greedy checks and develops the dynamic data structure needed for updates.

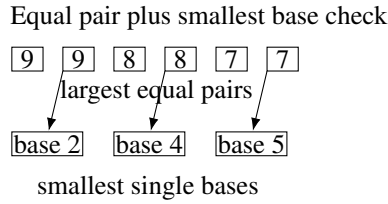


Figure 7. Check  $x$  triangles by pairing the  $x$  largest equal pairs with the  $x$  smallest bases.

### Subtask 1 – direct greedy or binary search

Greedyly form equal-length pairs, process pairs from largest to smallest, and complete each triangle with the largest available single base satisfying the triangle inequality. Equivalently, binary search the answer  $x$ : take the  $x$  largest equal-stick pairs and the  $x$  smallest remaining single sticks, sort them, and check the inequalities in order.

### Subtask 2 – process equal lengths in groups

The direct simulation repeats the same work for many equal-length sticks. Count pairs and singles by length and process each length as a group. This improves each snapshot to roughly linear in the number of different lengths.

### Subtask 3 – small updates

Maintain the current number  $x$  of triangles, together with used and unused singles and pairs in ordered sets. A segment tree stores the feasibility of the current matching. After an update, remove affected sticks, possibly decrease  $x$ , and greedily try to increase it again. Extra pairs after all singles are used can always make  $\lfloor 2p/3 \rfloor$  additional triangles.

### Subtask 4 – grouped dynamic version

Use the same method as subtask 3, but store counts by length instead of individual sticks. This keeps the update cost logarithmic in the number of represented lengths.

### Subtask 5 – full dynamic solution

Let  $o_1 < \dots < o_m$  be lengths with odd count. If  $k$  odd sticks are used, it is enough to use the  $k$  smallest. Define  $f(i)$  as the number of equal pairs that can pair with a single stick of length at most  $i$ . Hall's theorem gives the condition

$$f(o_i) + i \geq k + 1 \quad (1 \leq i \leq k).$$

A lazy segment tree stores the values  $f(o_i) + i$  while hiding non-odd indices with a large value. The maximum feasible  $k$  is found by binary search with range-minimum queries, or by a segment-tree walk. If  $p$  is the total number of equal pairs, the answer is

$$k + \left\lfloor \frac{2(p - k)}{3} \right\rfloor.$$

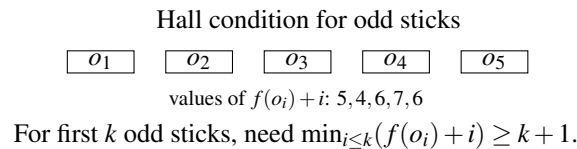


Figure 8. Segment tree view of the Hall condition for odd sticks.